

TinyKV Project 2 与 Project 3

代码变更及测试点分析

生成日期：2026-07-27

1. 分析范围与结论

本报告依据任务文档、Makefile、当前工作区相对 8ae6dd7 (Project1_PASS) 的实际代码差异以及测试源码，分析 Project 2 (RaftKV) 和 Project 3 (MultiRaftKV)。

Project 2 和 Project 3 的实现目前处于同一个未提交工作区中，两个项目在 raft/raft.go、raft/rawnode.go 和 kv/raftstore/peer_msg_handler.go 中存在连续演进关系，因此无法仅靠 Git 精确拆分每个项目各自的行数。本报告按功能语义归属拆分说明。

文件	增加	删除	所属功能
raft/log.go	158	17	Project 2: Raft 日志、压缩边界和快照
raft/raft.go	565	16	Project 2: 选举和复制; Project 3: 成员变更和领导权转移
raft/rawnode.go	91	8	Project 2: Ready 接口; Project 3: ConfChange 和 TransferLeader
kv/raftstore/peer_storage.go	107	9	Project 2: Ready 持久化、日志覆盖和快照应用
kv/raftstore/peer_msg_handler.go	515	5	Project 2: RaftStore 应用; Project 3: 变更成员和 Region Split
scheduler/server/cluster.go	44	1	Project 3: Region Heartbeat
scheduler/server/schedulers/balance_region.go	86	1	Project 3: Region 副本均衡

上述 7 个文件合计增加 1566 行、删除 57 行。

正式测试共 130 个：

- Project 2: 76 个，其中 2AA 24 个、2AB 26 个、2AC 2 个、2B 11 个、2C 13 个。
- Project 3: 54 个，其中 3A 16 个、3B 16 个、3C 22 个。

2. Project 2: RaftKV

2.1 目标与数据流

Project 2 把 Project 1 的单机 KV 改造成基于 Raft 的复制状态机。客户端请求经过以下路径：

1. RaftStorage 把 Get、Put、Delete 或 Snap 封装为 RaftCmdRequest；
2. Leader 把命令提议为 Raft 日志；

3. 日志复制到多数派后提交；
4. RaftStore 按日志顺序应用到 Badger；
5. 应用完成后通过 proposal callback 返回客户端；
6. Ready 中的 HardState、日志和快照在发送消息或推进状态前持久化。

2.2 raft/log.go 的改动与理由

1. 从 Storage 恢复日志

`newLog` 读取 `FirstIndex`、`LastIndex`、压缩边界 `term` 和未压缩 `entries`，初始化 `committed`、`applied`、`stabled` 以及 `dummy` 边界。

理由：节点重启后不能从空日志开始，必须从持久化日志和快照边界恢复一致的索引空间。

2. 区分 `committed`、`applied` 和 `stabled`

实现 `unstableEntries` 与 `nextEnts`，前者返回尚未持久化的日志，后者返回已提交但尚未应用的日志。

理由：Raft Ready 必须先持久化 `unstable entries`，再把 `committed entries` 交给状态机；三个进度不可混用。

3. 日志 `term`、切片和冲突覆盖

实现 `Term`、`lastTerm`、`matchTerm`、`slice` 和 `truncateAndAppend`。收到冲突日志时保留无冲突前缀，删除旧尾部，再追加 Leader 的日志，同时回退 `stabled`。

理由：这是 `AppendEntries` 的日志匹配属性基础，能确保不同节点最终形成相同日志。

4. 压缩边界与快照恢复

用 `firstIndex`、`dummyIndex`、`dummyTerm` 保留被压缩日志之前的比较点；`maybeCompact` 回收内存日志；`restore` 用快照索引和 `term` 重置日志进度。

理由：日志被 GC 后仍需边界上比较 `term`；落后过多的 Follower 需要通过快照跳过已删除日志。

2.3 raft/raft.go 的 Project 2 改动与理由

1. 节点初始化

从 Storage 恢复 `HardState` 和 `ConfState`，为每个 peer 建立 `Progress`，并恢复 `commit/applied`。

理由：`Term`、`Vote`、`Commit` 和成员列表是安全性状态，重启后必须延续，不能重新初始化。

2. 逻辑时钟与随机选举

Leader 按 `heartbeat timeout` 广播心跳；Follower/Candidate 按随机区间 [`electionTimeout`, `2*electionTimeout`) 发起选举。

理由：心跳维持领导权；随机选举超时降低多个节点同时竞选导致 `split vote` 的概率。

3. 角色转换和投票

实现 `Follower`、`Candidate`、`Leader` 转换，高 `term` 消息触发降级；投票同时检查“一任期一票”和候选日志是否至少同样新。

理由：任期单调和日志新旧判断共同防止旧节点成为 Leader，维护 Leader Completeness。

4. 日志复制

Leader 为每个 Follower 维护 `Match/Next`，发送包含 `prevIndex/prevTerm` 的 `Append`；拒绝时回退 `Next`，成功时推进 `Match/Next`。

理由：Leader 需要逐步定位双方最后一致的位置，再覆盖冲突后缀。

5. 多数派提交

对 Match 排序计算多数派索引，仅当该索引属于当前任期时推进 commit，并向其他节点广播新的提交位置。

理由：旧任期日志不能单独通过计数直接提交；当前任期日志被多数派复制后，之前的日志会随之安全提交。

6. 心跳和快照

心跳传播 commit 并确认 Leader 存活；当 Follower 的 Next 已落到压缩边界之前时发送快照，Follower 只接受比当前 committed 更新的快照。

理由：心跳兼具租约式存活通知和提交推进；快照让严重落后节点无需读取已经 GC 的日志。

2.4 rawnode.go 的 Project 2 改动与理由

实现 NewRawNode、Tick、Campaign、Propose、Step、Ready、HasReady 和 Advance。

Ready 汇总：

- SoftState: Leader 和角色变化；
- HardState: Term、Vote、Commit 变化；
- Entries: 待持久化日志；
- CommittedEntries: 待应用日志；
- Snapshot: 待持久化和应用快照；
- Messages: 待发送网络消息。

Advance 在上层处理 Ready 后推进 stabled/applied，清除已完成快照，并执行内存日志压缩。

理由：Raft 核心只负责状态机计算，上层负责磁盘和网络。Ready/Advance 明确了“先持久化、后发送、再推进”的交互边界。

2.5 peer_storage.go 的改动与理由

1. Append

把 Ready.Entries 写入 raftdb，删除被新日志覆盖的旧尾部，并更新 RaftLocalState 的 LastIndex/LastTerm。

理由：发生 Leader 更换时，未提交旧日志可能被覆盖，磁盘上的残留尾部必须同步删除。

2. SaveReadyState

在 WriteBatch 中处理 Snapshot、Entries、HardState 和 RaftLocalState。快照影响的 KV 元数据先落 kvdb，之后再写 raftdb。

理由：Ready 状态必须持久化后才能发送消息；快照写入顺序避免重启时观察到“Raft 已前进但状态机尚未安装”的状态。

3. ApplySnapshot

解析快照 Region，清理旧元数据和范围外数据，重建 RaftLocalState、RaftApplyState、RegionLocalState，调度 RegionTaskApply 并等待完成。

理由：快照不仅包含 Raft 索引，还代表完整状态机和 Region 元数据；必须整体替换并与后台快照文件应用同步。

2.6 peer_msg_handler.go 的 Project 2 改动与理由

1. HandleRaftReady

依次执行：获取 Ready、持久化、更新快照后的 StoreMeta、发送消息、应用 committed entries、Advance。

理由：持久化必须早于网络发送；状态机应用必须按 committed 顺序；Advance 只能在本轮 Ready 真正处理完后调用。

2. Proposal 与回调匹配

记录 proposal 的 index/term/callback；应用日志时按 index/term 匹配。被新 Leader 覆盖或错过的 proposal 返回 StaleCommand。

理由：客户端等待的是自己提议的那条日志，而不是相同索引上的其他任期日志。

3. 应用普通命令

检查 RegionEpoch 和 key range，在同一个 KV WriteBatch 中执行 Put/Delete 并更新 AppliedIndex；Get 和 Snap 在提交后读取快照并返回。

理由：状态机数据与 AppliedIndex 必须原子推进；Get 也经过 Raft，避免少数派上的旧 Leader 返回过时数据。

4. CompactLog

应用 CompactLog admin entry 时更新 TruncatedState，并调度异步 RaftLogGC 删除旧日志。

理由：先通过 Raft 一致地确定压缩点，再异步删除物理日志，可控制日志增长且不阻塞状态机应用。

3. Project 2AA: Leader Election (24 个)

序号	测试	作用
1	TestFollowerUpdateTermFromMessage2AA	验证 Follower 收到更高任期消息时更新 Term，并保持安全的 Follower 状态。
2	TestCandidateUpdateTermFromMessage2AA	验证 Candidate 遇到更高任期消息会放弃竞选并降级。
3	TestLeaderUpdateTermFromMessage2AA	验证 Leader 遇到更高任期消息会立即退位。
4	TestStartAsFollower2AA	验证新建节点从 Follower、无 Leader 和正确初始任期开始。
5	TestLeaderBcastBeat2AA	验证 Leader 的 heartbeat tick 向所有其他 peer 广播心跳。
6	TestFollowerStartElection2AA	验证 Follower 选举超时后自增任期、投自己并发送 RequestVote。
7	TestCandidateStartNewElection2AA	验证 Candidate 再次超时会开启更高任期的新一轮选举。
8	TestLeaderElectionInOneRoundRPC2AA	验证收到多数赞成票成为 Leader，票数不足时保持 Candidate。
9	TestFollowerVote2AA	验证每个任期最多投一票，可重复投给同一候选，但拒绝第二个候选。
10	TestCandidateFallback2AA	验证 Candidate 收到同任期或更高任期合法 Append 后承认 Leader 并降级。
11	TestFollowerElectionTimeoutRandomized2AA	验证 Follower 的选举超时落在随机区间内。
12	TestCandidateElectionTimeoutRandomized2AA	验证 Candidate 重选超时同样随机化。
13	TestFollowersElectionTimeoutNonconflict2AA	统计验证多个 Follower 的随机超时大多不会同时触发。
14	TestCandidatesElectionTimeoutNonconflict2AA	统计验证多个 Candidate 的重选时间冲突率受控。
15	TestLeaderElection2AA	在节点失效和网络条件组合下验证能否按多数派规则选出 Leader。
16	TestLeaderCycle2AA	验证多轮 Leader 更换中角色和任期持续正确推进。
17	TestVoteFromAnyState2AA	验证任意角色收到更高任期 RequestVote 都按任期规则处理。
18	TestSingleNodeCandidate2AA	验证单节点集群竞选时凭自身一票立即成为 Leader。
19	TestCandidateResetTermMessageType_MsgHeartbeat2AA	验证 Candidate 收到合法 Leader 心跳后同步任期并成为 Follower。
20	TestCandidateResetTermMessageType_MsgAppend2AA	验证 Candidate 收到合法 Append 后同步任期并成为 Follower。
21	TestDisruptiveFollower2AA	验证隔离 Follower 任期升高后，其消息会迫使旧 Leader 按 Raft 规则退位。
22	TestRecvMessageType_MsgBeat2AA	验证只有 Leader 响应本地 MsgBeat，Follower/Candidate 不错误广播。
23	TestCampaignWhileLeader2AA	验证 Leader 收到本地竞选触发时不自增任期或离开 Leader 状态。
24	TestSplitVote2AA	验证一次平票后节点能在下一随机轮次完成选举。

4. Project 2AB: Log Replication (26 个)

序号	测试	作用
1	TestLeaderStartReplication2AB	验证新 Leader 追加本任期空日志并开始向 Follower 复制。
2	TestLeaderCommitEntry2AB	验证日志达到多数副本后 Leader 推进 commit。
3	TestLeaderAcknowledgeCommit2AB	验证 AppendResponse 正确推进目标 Follower 的 Match/Next。
4	TestLeaderCommitPrecedingEntries2AB	验证当前日志提交时，其之前尚未提交的日志一并提交。
5	TestFollowerCommitEntry2AB	验证 Follower 按 LeaderCommit 推进本地 commit，但不超过已有日志。
6	TestFollowerCheckMessageType_MsgAppend2AB	验证 Follower 对 prevIndex/prevTerm 不匹配的 Append 拒绝。
7	TestFollowerAppendEntries2AB	验证匹配位置后的新日志被正确追加，冲突后缀被替换。
8	TestLeaderSyncFollowerLog2AB	验证 Leader 通过回退 Next 修复 Follower 的缺失或冲突日志。
9	TestVoteRequest2AB	验证 RequestVote 携带候选人的最后日志 index/term。
10	TestVoter2AB	验证投票者只支持日志至少与自己同样新的候选人。
11	TestLeaderOnlyCommitsLogFromCurrentTerm2AB	验证 Leader 不直接用多数计数提交旧任期日志，只提交当前任期索引。
12	TestProgressLeader2AB	验证成为 Leader 后每个 peer 的 Match/Next 初始化正确。
13	TestLeaderElectionOverwriteNewerLogs2AB	验证提交链安全地覆盖少数派节点上看似更新但未提交的日志。
14	TestLogReplication2AB	验证连续 proposal 在整个集群中以相同顺序复制并提交。
15	TestSingleNodeCommit2AB	验证单节点 Leader 可立即提交自身日志。
16	TestCommitWithoutNewTermEntry2AB	验证新 Leader 在没有本任期条目时不能错误提交旧任期日志。
17	TestCommitWithHeartbeat2AB	验证心跳/后续消息能把最新 commit 传播给 Follower。
18	TestDuellingCandidates2AB	验证多个候选竞争时仍遵守 term、投票和多数派规则。
19	TestCandidateConcede2AB	验证 Candidate 收到合法 Leader 复制消息后让步并接受日志。
20	TestOldMessages2AB	验证旧任期 Leader 的迟到消息不会覆盖新任期已形成的日志。
21	TestProposal2AB	验证 Leader 接受 proposal，非 Leader 不在本地错误追加。
22	TestHandleMessageType_MsgAppend2AB	验证各角色收到 Append 时的状态转换和日志处理。
23	TestRecvMessageType_MsgRequestVote2AB	验证各角色对 RequestVote 的任期和日志新旧判断一致。
24	TestAllServerStepdown2AB	验证所有角色收到更高任期消息后都能退回 Follower。
25	TestHeartbeatUpdateCommit2AB	验证 Heartbeat 中的 commit 能推进 Follower 的提交索引。
26	TestLeaderIncreaseNext2AB	验证 Leader 发送/确认日志后正确增长 Progress.Next，避免重复或跳跃。

5. Project 2AC: RawNode (2 个)

序号	测试	作用
1	TestRawNodeStart2AC	验证新启动 RawNode 的首个 Ready 包含正确 SoftState、HardState、日志和已提交项，并能 Advance。
2	TestRawNodeRestart2AC	验证从持久化 Storage 重启后只暴露尚需处理的 Ready 内容，不重复稳定日志。

6. Project 2B: RaftKV 集成测试 (11 个)

序号	测试	作用
1	TestBasic2B	单客户端正常网络下验证 Put/Get/Delete 和复制状态机基本链路。
2	TestConcurrent2B	多客户端并发写入，验证日志顺序、回调和最终数据一致性。
3	TestUnreliable2B	多客户端加丢包/延迟，验证重试下仍保持正确数据。
4	TestOnePartition2B	验证多数派分区可继续服务、少数派旧 Leader 不能提交，网络恢复后日志追平。
5	TestManyPartitionsOneClient2B	单客户端、多轮网络分区下验证可用性和线性一致结果。
6	TestManyPartitionsManyClients2B	多客户端、多轮分区下验证并发一致性。
7	TestPersistOneClient2B	单客户端配合服务器重启，验证 Raft 与 KV 状态持久化恢复。
8	TestPersistConcurrent2B	多客户端和重启组合，验证并发日志不会在恢复后丢失或重放错误。
9	TestPersistConcurrentUnreliable2B	在并发、重启和不可靠网络同时存在时验证安全性。
10	TestPersistPartition2B	在并发重启和网络分区下验证多数派提交与恢复。
11	TestPersistPartitionUnreliable2B	综合丢包、重启、分区和多客户端的 Project 2B 压力场景。

7. Project 2C: Snapshot 与 Log GC (13 个)

7.1 Raft 层 (7 个)

序号	测试	作用
1	TestRestoreSnapshot2C	验证比 committed 更新的快照重置日志、commit、applied 和成员集合。
2	TestRestoreIgnoreSnapshot2C	验证旧快照或不前进的快照被忽略。
3	TestProvideSnap2C	验证 Follower 落后到压缩边界之前时 Leader 改发 Snapshot。
4	TestRestoreFromSnapMsg2C	验证收到 MsgSnapshot 后恢复状态并回复正确的 AppendResponse。
5	TestRestoreFromSnapWithOverlappingPeersMsg2C	验证快照 ConfState 与原成员重叠时仍正确重建 Progress。
6	TestSlowNodeRestore2C	验证严重落后节点通过快照追上后能继续接收新日志。
7	TestRawNodeRestartFromSnapshot2C	验证 RawNode 从包含快照的 Storage 重启时 Ready 和索引边界正确。

7.2 RaftStore 集成层（6 个）

序号	测试	作用
8	TestOneSnapshot2C	隔离节点并触发日志 GC，验证其通过快照恢复 Put/Delete、重启后数据仍在且日志已截断。
9	TestSnapshotRecover2C	单客户端、持续快照和服务器重启下验证恢复。
10	TestSnapshotRecoverManyClients2C	多客户端、快照和重启下验证一致性及日志长度受控。
11	TestSnapshotUnreliable2C	不可靠网络和快照并存时验证状态机最终一致。
12	TestSnapshotUnreliableRecover2C	不可靠网络、快照、重启组合测试。
13	TestSnapshotUnreliableRecoverConcurrentPartition2C	综合并发客户端、丢包、重启、分区、快照和 Log GC 的 Project 2 最强集成场景。

8. Project 3: MultiRaftKV

8.1 raft/raft.go 与 rawnode.go 的 Project 3 改动

1. 成员变更

新增 PendingConfIndex，Leader 同一时间只允许一个尚未应用的 ConfChange。RawNode 支持 ProposeConfChange 和 ApplyConfChange；addNode/removeNode 更新 Progress 和 quorum。

理由：成员变化会改变多数派定义；若多个配置变更未按顺序应用，可能对同一日志使用不同 quorum，破坏安全性。

2. Leadership Transfer

Leader 记录 leadTransferee：

- 目标已追平时发送 MsgTimeoutNow；
- 目标落后时先发送 Append 追日志；
- 转移期间拒绝新 proposal；
- 超时、目标被删除或更高 term 到来时取消转移；
- Follower 收到 MsgTimeoutNow 后立即竞选。

理由：目标必须拥有最新日志才能安全接任；暂停新 proposal 防止目标永远追不上当前 Leader。

8.2 peer_msg_handler.go 的 Project 3 改动

1. 请求前置检查

检查 StoreID、PeerID、Term、RegionEpoch、Leader 身份和 key range；错误返回 NotLeader、EpochNotMatch 或 KeyNotInRegion。

理由：MultiRaft 中请求可能带过期路由，不能提交到错误 Region 或已过期 Peer。

2. TransferLeader 与 ChangePeer

TransferLeader 直接调用 RawNode，不写普通 Raft 日志。ChangePeer 编码为 EntryConfChange，提交后更新 Region peers、ConfVer、PeerCache、StoreMeta 和 RegionLocalState，并处理删除自身。

理由：转移领导权是 Raft 控制动作；成员变更则必须被所有副本以相同顺序提交和应用。

3. Region Split

校验 split key、epoch、新 Region/Peer ID 数量；把旧 Region 切成相邻的左右区间并增加 Version，写入两份 RegionLocalState，创建和注册新 Peer，更新 regionRanges 并触发 Scheduler heartbeat。

理由：Range 分片要求区间无重叠、无空洞，且元数据、路由和本地 Peer 必须原子地转到新布局。

4. Snapshot 后的 StoreMeta

应用 Snapshot 后替换旧 Region range，并更新 regions/regionRanges。

理由：快照可能改变 Region epoch、成员或范围；内存路由必须与持久化元数据同步。

8.3 cluster.go 的改动与理由

processRegionHeartbeat:

- 拒绝缺失 Region/Epoch 的心跳；
- 对同 ID Region 比较 Version 和 ConfVer；
- 对重叠 Region 检查 epoch，拒绝孤立节点发来的过期信息；
- PutRegion 后收集新旧及被覆盖 Region 涉及的 Store；
- 重新计算这些 Store 的 leader、region、pending peer 数量和 size。

理由：Scheduler 的决策依赖可信的全局 Region 树和 Store 统计；接受过期心跳可能让已 split/变更的 Region 重新覆盖新布局。

8.4 balance_region.go 的改动与理由

Balance Region Scheduler:

1. 过滤 Offline、Down 时间过长的 Store；
2. 按 RegionSize 从大到小排列 source；
3. 按 Pending Region、Follower Region、Leader Region 的优先级选择待移动 Region；
4. 目标 Store 必须不已有该 Region 副本且 size 更小；
5. 只有 source-target 差值至少为 Region 大小两倍时才移动；
6. 分配新 Peer 并创建 MovePeer Operator。

理由：优先迁移 pending peer 有助于缓解异常节点；两倍阈值避免副本来回震荡；排除已有副本保证同一 Region 不在同一 Store 重复放置。

9. Project 3A: ConfChange 与 Leader Transfer (16 个)

序号	测试	作用
1	TestAddNode3A	验证添加节点后 Progress、成员集合和新 quorum 正确。
2	TestRemoveNode3A	验证删除节点后成员集合与 quorum 更新，重复删除安全。
3	TestCommitAfterRemoveNode3A	验证移除节点改变多数派后，已有日志能按新配置正确提交。
4	TestLeaderTransferToUpToDateNode3A	验证日志已追平的目标收到 TimeoutNow 并接任 Leader。
5	TestLeaderTransferToUpToDateNodeFromFollower3A	验证向 Follower 发起的转移请求能转发给当前 Leader 并完成。
6	TestLeaderTransferToSlowFollower3A	验证目标落后时先补日志，追平后再触发选举。
7	TestLeaderTransferAfterSnapshot3A	验证目标需要 Snapshot 才能追平时仍可完成领导权转移。
8	TestLeaderTransferToSelf3A	验证转移给自己是安全无操作。
9	TestLeaderTransferToNonExistingNode3A	验证不存在于成员列表的目标被忽略。
10	TestLeaderTransferReceiveHigherTermVote3A	验证转移期间收到更高任期消息会按 Raft 规则退位并取消转移。
11	TestLeaderTransferRemoveNode3A	验证转移目标被移除时中止转移，不保留悬空状态。
12	TestLeaderTransferBack3A	验证转移过程中反向请求不会形成循环或破坏状态。
13	TestLeaderTransferSecondTransferToAnotherNode3A	验证新的转移目标能替换或正确处理正在进行的旧目标。
14	TestTransferNonMember3A	验证本节点已被移出配置时，即使收到 TimeoutNow 和投票也不会错误成为 Leader。
15	TestRawNodeProposeAndConfChange3A	验证 RawNode 提议、提交并应用 Add/Remove ConfChange 的完整接口。
16	TestRawNodeProposeAddDuplicateNode3A	验证重复 AddNode 幂等，不产生重复成员。

10. Project 3B: RaftStore ConfChange 与 Split (16 个)

序号	测试	作用
1	TestTransferLeader3B	在五个 Store 间连续转移同一 Region 的 Leader，验证端到端控制命令。
2	TestBasicConfChange3B	反复增删 Peer，验证快照补数据、ConfVer 增长、移除节点清数据及 Peer ID 更新。
3	TestConfChangeRemoveLeader3B	验证移除当前 Leader 后重新选举，原 Store 停止接收数据，重新加入后能追平。
4	TestConfChangeRecover3B	单客户端、成员变更和服务器重启组合测试。
5	TestConfChangeRecoverManyClients3B	多客户端、成员变更和重启组合测试。
6	TestConfChangeUnreliable3B	不可靠网络下反复成员变更和并发请求。
7	TestConfChangeUnreliableRecover3B	不可靠网络、成员变更、并发和重启组合。
8	TestConfChangeSnapshotUnreliableRecover3B	在上一场景加入 Snapshot/Log GC，验证新 Peer 和落后 Peer 恢复。
9	TestConfChangeSnapshotUnreliableRecoverConcurrentPartition3B	综合分区、丢包、重启、快照、成员变化和并发客户端。
10	TestOneSplit3B	触发一次 Region Split，验证左右边界、不同 Region ID、旧 Region 的 KeyNotInRegion 和隔离节点追平。
11	TestSplitRecover3B	单客户端、自动 Split 和服务器重启组合。
12	TestSplitRecoverManyClients3B	多客户端、自动 Split 和重启组合。
13	TestSplitUnreliable3B	不可靠网络下自动 Split 和多客户端一致性。
14	TestSplitUnreliableRecover3B	不可靠网络、Split、并发和重启组合。
15	TestSplitConfChangeSnapshotUnreliableRecover3B	同时启用 Split、ConfChange、Snapshot、丢包和重启。
16	TestSplitConfChangeSnapshotUnreliableRecoverConcurrentPartition3B	Project 3B 最强综合测试：再加入持续网络分区和并发客户端。

11. Project 3C: Scheduler (22 个)

11.1 Region Heartbeat (18 个)

序号	测试	作用
1	TestRegionNotUpdate3C	验证重复发送完全相同的 Region heartbeat 不破坏缓存。
2	TestRegionUpdateVersion3C	验证更高 Version 的 Region 信息可更新。
3	TestRegionWithStaleVersion3C	验证 Version 落后的 heartbeat 即使 ConfVer 较高也被判 stale。
4	TestRegionUpdateVersionAndConfVer3C	验证 Version 和 ConfVer 同时提高时正常更新。
5	TestRegionWithStaleConfVer3C	验证 ConfVer 落后的 heartbeat 被拒绝。
6	TestRegionAddPendingPeer3C	验证新增 pending peer 的 Region 信息被记录并关联 Store 统计。
7	TestRegionRemovePendingPeer3C	验证 pending peer 消失后 Store pending 统计归零。
8	TestRegionRemovePeers3C	验证 heartbeat 中 Peer 减少会更新 Region 副本集合和 Store 统计。
9	TestRegionAddBackPeers3C	验证被移除的 Peer 再加入时缓存能恢复完整成员。
10	TestRegionChangeLeader3C	验证 Leader 变化会更新 Region 和各 Store leader 统计。
11	TestRegionChangeApproximateSize3C	验证 ApproximateSize 变化会更新 Region 和 Store size。
12	TestRegionCounts3C	验证每个 Store 的 RegionCount 等于 Region 树中的实际副本数。
13	TestRegionGetRegions3C	验证按列表和元数据接口取得的 Region 与 heartbeat 内容一致。
14	TestRegionGetStores3C	验证可正确查询 Region 的全部 Store 和非 Leader Store。
15	TestRegionGetStoresInfo3C	验证 Store 的 leader/region 数量和 size 与 Region 索引统计一致。
16	TestHeartbeatSplit3C	验证 Split 后左右 Region heartbeat 无论到达顺序如何，都构建正确 key range，过渡期不伪造缺失范围。
17	TestRegionSplitAndMerge3C	多轮 Split/Merge 后验证 Region 树按 ID 和 key 查询始终正确且重叠项被替换。
18	TestConcurrentHandleRegion3C	多 Store 并发发送 Region heartbeat，验证锁和缓存更新无竞态崩溃。

11.2 Balance Region (4 个)

序号	测试	作用
19	TestReplicas13C	单副本场景验证从重 Store 移到最轻可用 Store，Offline Store 被过滤且调度限制生效。
20	TestReplicas33C	三副本场景验证目标不能已有副本、Down Store 被过滤、无合适目标时不生成 Operator。
21	TestReplicas53C	五副本及复杂 Store size 分布下验证 source/target 选择和副本放置约束。
22	TestReplacePendingRegion3C	验证优先选择含 pending peer 的 Region，并移动到唯一未持有该 Region 的合适 Store。

12. 测试力度评价

12.1 Project 2

Project 2 的测试力度很强：

- 2AA/2AB 对任期、投票、角色、日志冲突、Progress 和 commit 规则进行细粒度状态机测试；
- 2B 使用真实 RaftStore、多个 Store 和模拟网络，覆盖并发、分区、丢包与重启；
- 2C 验证快照、Log GC、严重落后节点以及多故障组合；
- 多项随机和综合测试能发现只在少数消息顺序下出现的问题。

局限：

- 模拟网络仍不等同于真实网络、磁盘损坏或进程在任意指令点崩溃；
- 未系统执行长时间 soak、race detector、模糊消息序列和磁盘故障注入；
- Makefile 中部分 2B/2C 命令带 `|| true`，脚本整体退出码可能掩盖单项失败，必须查看每个测试输出。

12.2 Project 3

Project 3 同样覆盖较强：

- 3A 对领导权转移的慢节点、快照、目标删除、重复转移等边界覆盖细致；
- 3B 把 ConfChange、Split、Snapshot、Restart、Partition 和并发交叉组合；
- 3C 验证 stale heartbeat、Region 重叠替换、Store 统计和副本均衡选择。

局限：

- 调度测试以确定性 mock cluster 为主，未覆盖长期多 Scheduler 竞争和频繁状态抖动；
- 简化 ConfChange 不是 Raft Joint Consensus，只支持一次一个节点的变更模型；
- Split 主要覆盖元数据和恢复，未穷举所有 snapshot overlap、重复 admin request 和极端 key range；
- 通过固定测试不能形式化证明不存在死锁、数据竞态或隐藏的安全性问题。

13. 最终结论

Project 2 已建立完整的 Raft 复制状态机链路：选举、日志复制、持久化、状态机应用、回调、快照和日志回收。Project 3 在此基础上加入成员变化、领导权转移、Region Range 分片、过期路由防护以及 Scheduler 的心跳缓存和副本均衡。

130 个正式测试点从纯 Raft 状态机一直覆盖到多 Store 故障组合，课程验收强度高。全部通过时，可以认为实现满足 Project 2/3 的主要设计要求；但仍应结合逐项退出状态、重复随机运行、`go test -race`、崩溃注入和更长时间压力测试判断工程可靠性。